ARTIFICIAL INTELLIGENCE CSA1701 LAB OBSERVATON
Mohammed Fareeth S A (192511288)

EXPERIMENT 7:

Aim

To implement and solve the classical AI Vacuum World problem using the Breadth-First Search (BFS) graph search algorithm to find the optimal sequence of actions from an initial state to the goal state.

Algorithm

1. Define State Space: Represent each state as a tuple containing the agent's location and the dirt status of all rooms (e.g., (location, status_a, status_b)).
2. Initialize Frontier and Explored Set:
    - Create a queue (frontier) containing the starting state and the path (list of actions) taken to reach it.
    - Initialize an empty set for explored states to prevent redundant paths and cycles.
3. Queue Processing (BFS Loop):
    - While the frontier is not empty, pop the oldest state and its corresponding path from the queue.
    - If the current state satisfies the goal condition (all rooms are clean), return the path.
4. State Expansion:
    - If the state has not been explored, mark it as explored.
    - Generate all possible legal successor states based on valid actions (Left, Right, Suck) and add them to the queue along with the updated path
5. Terminate: Repeat until the goal state is found and output the optimal action sequence.

CODE:

```
from collections import deque

class VacuumBFS:
    def __init__(self, initial_location, initial_a, initial_b):
        self.initial_state = (initial_location, initial_a, initial_b)

    def is_goal(self, state):
        _, status_a, status_b = state
        return not status_a and not status_b

    def get_successors(self, state):
        location, status_a, status_b = state
        successors = []

        if location == 'A':
            successors.append(('Suck', ('A', False, status_b)))
            successors.append(('Right', ('B', status_a, status_b)))
        else:
            successors.append(('Suck', ('B', status_a, False)))
            successors.append(('Left', ('A', status_a, status_b)))

        return successors

    def search(self):
        frontier = deque([(self.initial_state, [])])
        explored = set()

        while frontier:
            current_state, path = frontier.popleft()

            if self.is_goal(current_state):
                return path

            if current_state not in explored:
                explored.add(current_state)
                for action, next_state in self.get_successors(current_state):
                    if next_state not in explored:
                        frontier.append((next_state, path + [action]))

        return None

problem = VacuumBFS(initial_location='A', initial_a=True, initial_b=True)
optimal_path = problem.search()

print(f"Initial State: ('A', True, True)")
print("Goal State Reached!\n")
print("Optimal Action Sequence:")
for i, action in enumerate(optimal_path, 1):
    print(f"{i}. {action}")

print(f"\nTotal Cost (Steps): {len(optimal_path)}")
```

Output:

```
============== RESTART: C:/Users/Admin/OneDrive/Documents/aiexp7.py
Initial State: ('A', True, True)
Goal State Reached!

Optimal Action Sequence:
1. Suck
2. Right
3. Suck

Total Cost (Steps): 3
```

Result

The BFS algorithm successfully explored the state space starting from a state where the agent is at room *A* and both rooms are dirty ('A', True, True). By systematically checking level-by-level, the algorithm identified the shortest sequence of actions required to reach the goal state where all rooms are clean.

Conclusion

The program successfully demonstrated the application of Breadth-First Search to the Vacuum World problem. Unlike simple reflex agents which can wander or lack global planning, BFS guaranteed the discovery of the shortest and most optimal path to achieve the goal state by exhaustively exploring the state space level-by-level.